4. The *agent* processes the search results, noting key information about flights, accommodations, and attractions.
5. The *agent* returns to the decision-making phase. It may decide it needs more specific information and choose to use a flight price comparison tool.
6. The *agent* uses the *flight comparison* tool to find suitable flights within the budget.
7. The *agent* notes the flight options and prices.
8. This process continues with the *agent* attempting to use other tools, such as
   - A *hotel booking* tool to find family-friendly accommodations
   - A *currency conversion* tool to ensure budget calculations are accurate
   - A *weather forecast* tool to recommend appropriate activities based on projected weather reports
9. After several iterations of using tools and making observations, the *agent* determines it has enough information to create a vacation plan.
10. The *agent* compiles all the gathered information into a coherent seven-day itinerary, including flight details, hotel recommendations, daily activities, and a budget breakdown. This final plan is presented to the user.

With all the information presented to the user, they will be able to ask additional queries to improve the results, which may also trigger additional iterations on the *agent* side. But as far as the user is concerned, the AI interface is abstracted from them.

6.4.2 Creating an agent using LangChain.js

Let's now look at an example of how to create an agent using LangChain and then how to integrate this with the Vercel AI SDK into our application. To create an agent capable of calling tools to achieve a specific goal, follow the basic steps outlined here. In LangChain, agents are essentially chains, as they are instances of the `Runnable` abstract class. This design allows them to be integrated into other chain compositions seamlessly.

To successfully create an agent, you will need the following three components:

- *A list of tools*—This includes the various tools your agent can utilize to perform tasks.
- *The AI model*—Specify which AI model your agent will use for processing and decision making.
- *The system prompt*—Define the system prompt that guides the agent's behavior and responses.

Having already covered how to define a system prompt and select an AI model, the next step is to establish your list of tools.

To create a tool, you can either use a list of available community tools from https://js.langchain.com/docs/integrations/tools/ or define a custom tool using the `tool` function from the `@langchain/core/tools` package. Once you have configured your tools, you can proceed to create an instance of your agent. The following listing shows how to instantiate an agent for integration into other chains.